

Attach an Application Load Balancer to EC2 Auto Scaling Group

Getting Started

What is a load balancer?

Amazon EC2 offers Elastic Load Balancing to distribute incoming traffic over multiple running EC2 instances in one or more availability zones. It also keeps track of the health of the instances registered with it and only sends traffic to those that pass the configured health checks. A **load balancer** serves as the single point of contact for clients. After the load balancer receives a request, it evaluates the listener rules to determine which rule to apply and then selects a registered instance to redirect the request.

AWS offers several different types of load balancers, including application load balancers and network load balancers. An **application load balancer** functions at the application layer (Layer 7 of the OSI model), while a **network load balancer** functions at the network layer (Layer 4).

In this lab, we'll explore the workings of application load balancers by configuring an Auto Scaling group (ASG) to work with an application load balancer. We'll start by creating and configuring the load balancer and target group, then create a launch template for the Auto Scaling group. We'll then create and configure the Auto Scaling group and attach it to our load balancer. Finally, we'll run a few tests on our application load balancer and monitor our instances.

Create Security Groups

A security group is a set of firewall rules that control inbound and outbound traffic to and from our EC2 resources. In our case, we want to control the traffic to and from our application load balancer, as well as the instances within our Auto Scaling group.

In this task, we'll create two security groups, one for the instance launch template, which will later be used by the Auto Scaling group to launch new instances, and one for our application load balancer.

After completing this task, the provisioned infrastructure will be similar to the one shown in the figure below:



Load balancer security group

Follow the steps below to create the security group for the application load balancer:

1. Search for "EC2" on the AWS Management Console, then click the "EC2" service from the search results.
2. From the sidebar, click "Security Groups" under "Network & Security."
3. Click the "Create security group" button.
4. Name the security group `LoadBalancerSecurityGroup` and set the description to "Allows HTTP access."
5. Keep the default VPC selected.
6. Click the "Add rule" button under the "Inbound rules" section to add a new inbound rule. Configure the new rule as follows:
 1. Select `HTTP` from the "Type" drop-down. Doing so automatically sets the port range to `80`.
 2. Select `Anywhere-IPv4` from the "Source" drop-down. Doing so sets the IP address range to `0.0.0.0/0`.
7. Click the "Create security group" button.

With these configurations, we've defined a single rule for our load balancer: HTTP to communicate with the EC2 instances.

Launch template security group

Follow the steps below to create the security group for the instance launch template:

1. From the "Network & Security > Security Groups" page, click the "Create security group" button.
2. Name the security group `LaunchTemplateSecurityGroup` and set the description to "Allows HTTP and SSH access."
3. Keep the default VPC selected.
4. Click the "Add rule" button under the "Inbound rules" section to add a new inbound rule. Configure the rule as follows:
 1. Select `HTTP` from the "Type" drop-down. Doing so automatically sets the port range to `80`.

2. Select **Anywhere-IPv4** from the “Source” dropdown. Doing so sets the IP address range to **0.0.0.0/0**.
5. Add another inbound rule and configure it as follows:
 1. Select **SSH** from the “Type” dropdown. Doing so automatically sets the port range to **22**.
 2. Select **Anywhere-IPv4** from the “Source” drop-down. Doing so sets the IP address range to **0.0.0.0/0**.
6. Click the “Create security group” button.

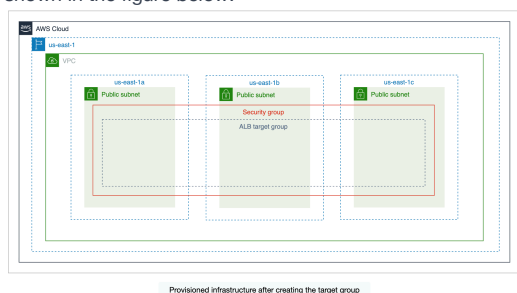
With these configurations, we’ve defined two traffic rules for our EC2 instances: SSH, for connecting to the instance, and HTTP, for communicating with the resources configured behind the load balancer and serving the application.

Create a Target Group

Load balancers direct incoming traffic to different instances, called **targets**, based on the request attributes. The instances that serve as targets form a logical grouping called a **target group**.

Apart from routing the incoming requests, the target group is also responsible for performing health checks on the instances registered as targets. It keeps track of the health of the instances and ensures that any unhealthy targets are taken out of service. When attached to an Auto Scaling group, the ASG replaces the unhealthy instances with new ones.

In this task, we’ll create a target group for our application load balancer. After completing this task, the provisioned infrastructure will be similar to the one shown in the figure below:



Configure target group details

Follow the steps below to create the target group:

1. From the sidebar, click “Target Groups” under “Load Balancing.”
2. Click the “Create target group” button.
3. Ensure that “Instances” is selected under “Choose a target type.” This ensures traffic is forwarded to EC2 instances created within our specified VPC. In our case, we’re using the default VPC in the **us-east-1** region.
4. Set the name of the target group to **LoadBalancerTargetGroup**.
5. Ensure that **HTTP** is selected under “Protocol” and the port is set to **80**. This specifies the protocol and port that the application load balancer uses to route traffic to the targets registered within the target group.
6. Ensure that the default VPC is selected under “VPC.” All our instances will be launched within the same VPC.

Configure health checks

Under the “Health checks” section, we can define custom health checks to keep track of the status of our application. For the purpose of this Cloud Lab, we’ll use the default health check configurations. Regardless, let’s look into this section in detail to see how custom health checks can be configured.

“Health check protocol” defines the protocol used to carry out the checks. By default, the protocol is **HTTP**. “Health check path” defines the path that is pinged when conducting a health check. The default value of **/** defines that the root address of the target is pinged.

Next, expand the “Advanced health check settings” section. Here, we can configure several thresholds and timeouts for the health checks, including the number of health checks that an instance must fail before being considered unhealthy, the time between consecutive health checks, and others.

Since we’re sticking with the default configurations, we don’t need to change anything in this section. The default configurations define that the root address of the instance is pinged once every thirty seconds, and anything other than a **200** response from the instance is considered a failed health check. After two consecutive failed health checks, the instance will be marked unhealthy.

Click “Next” to proceed to the next step.

Register targets

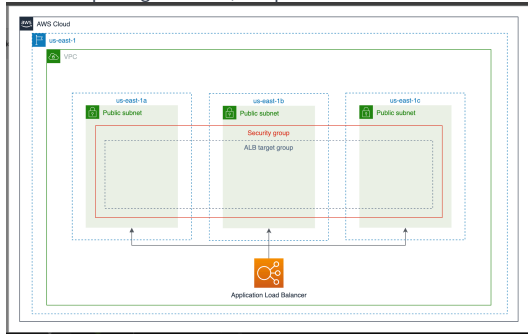
The “Register targets” page allows us to register EC2 instances within our chosen VPC as targets where traffic will be routed. Since we have yet to launch any instances, we don’t want to register any targets for now.

Simply click the “Create target group” button at the end of the page.

Create an Application Load Balancer

With the security and target groups configured, we can move on to creating our application load balancer. When a client accesses the load balancer, the ALB analyzes their request and redirects it to a target registered within the target group.

After completing this task, the provisioned infrastructure will be similar to the one shown in the figure below:



Follow the steps below to create the application load balancer:

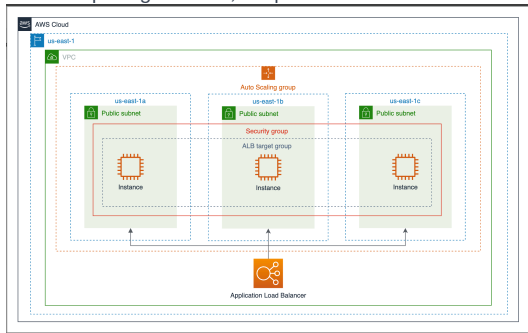
1. From the sidebar, click “Load Balancers” under the “Load Balancing” option.
2. Click the “Create load balancer” button.
3. Since we want to create an application load balancer, click the “Create” button under the “Application Load Balancer” option.
4. Name the load balancer `MyLoadBalancer`.
5. Ensure that “Internet-facing” is selected under “Scheme.” Since we want external clients to be able to ping the load balancer, we won’t be choosing the “Internal” option.
6. Ensure that “IPv4” is selected under “IP address type.”
7. Keep the default VPC selected. We must create the load balancer in the same VPC where the instances will be launched.
8. Select all availability zones under “Mappings.” Each mapping corresponds to a separate subnet in a different availability zone. The load balancer will route traffic to targets within the selected availability zone and subnet.
9. Under “Security groups,” deselect the default security group and select `LoadBalancerSecurityGroup` from the drop-down menu.
10. Under the “Listeners and routing” section, ensure that “HTTP” is selected under “Protocol” and the port is set to `80`.
11. Select `LoadBalancerTargetGroup` from the “Select a target group” drop-down.
12. Click the “Create load balancer” button.

Click “View load balancer” to return to the “Load Balancers” page where our newly created load balancer should be visible in the list.

Create an Auto Scaling Group

In this task, we’ll create an instance launch template, as well as an Auto Scaling group, and attach it to our load balancer.

After completing this task, the provisioned infrastructure will be similar to the one shown in the figure below:



Create a launch template

1. From the sidebar, click “Launch Templates” under “Instances.”
2. Click the “Create launch template” button.
3. Name the launch template `AutoScalingLaunchTemplate`.
4. Under the “Application and OS Images (Amazon Machine Image)” section, select the “Amazon Linux” option from the list of AMIs and select the “Amazon Linux 2023” AMI.
5. Under “Instance type” select `t2.micro` from the drop-down.
6. Under “Network settings,” select the “Select existing security group” option under “Firewall (security groups).”
7. Select `LaunchTemplateSecurityGroup` from the “Security groups” drop-down.

8. Expand the “Advanced details” section at the end and enter the following script in the “User data” field. Finally, click “Create launch template.”

```
#!/bin/bash

# Enabling admin rights
sudo su

# Updating system and installing httpd
yum update -y
yum install -y httpd.x86_64

# Launching a simple httpd server
systemctl start httpd.service
systemctl enable httpd.service

# Writing HTML file to serve on the httpd server
echo "Hello World from $(hostname)" > /var/www/html/index.html
```

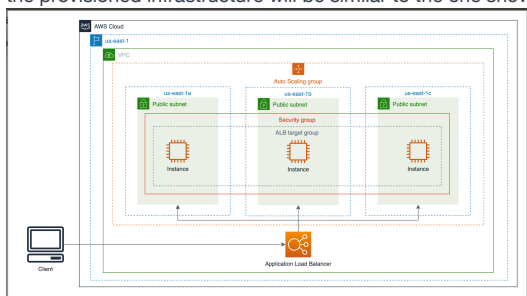
Create an Auto Scaling group

1. From the sidebar, click “Auto Scaling Groups” under the “Auto Scaling” option.
2. Click the “Create an Auto Scaling group” button.
3. Name the Auto Scaling group MyAutoScalingGroup.
4. Under “Launch template,” select AutoScalingLaunchTemplate and click “Next” to proceed.
5. Ensure the default VPC is selected under the “Network” section.
6. From the “Availability Zones and subnets” drop-down, select every availability zone. Click “Next” to proceed.
7. Under “Load balancing,” select “Attach to an existing load balancer.” Select LoadBalancerTargetGroup from the drop-down.
8. Under “Health checks,” enable ELB health checks and click “Next” to proceed.
9. Under “Group size,” set the capacities as follows:
 1. Desired capacity: 2
 2. Min desired capacity: 2
 3. Max desired capacity: 3
10. Click “Skip to review” to skip the remaining steps. Click “Create an Auto Scaling group.”

Putting It All Together

In this task, we'll carry out a few tests on our application load balancer to verify that it is working as expected. We'll also explore the instance health monitoring options offered by the target group.

Although we won't be creating any new resources in this task, we will make a request to our load balancer and carry out some tests. After completing this task, the provisioned infrastructure will be similar to the one shown in the figure below:



Target group monitoring

With the Auto Scaling group created, we have a couple of running instances. These instances should automatically be registered as targets within our target group. Let's go ahead and verify this:

1. From the sidebar, click “Target Groups” under the “Load Balancing” option.
2. Click LoadBalancerTargetGroup from the list to open its details page.

The “Registered targets” section lists all instances currently registered as targets under the target group. Since we've attached the load balancer to an Auto Scaling group, all instances launched by the Auto Scaling group will automatically be registered as targets.

The “Health status” column displays the current condition of each instance based on the health checks we configured when setting up the load balancer. This status may take one of six values: `initial`, `healthy`, `unhealthy`, `unused`, `draining`, and `unavailable`. While the instances are being launched and registered, the health status is `initial`, which changes to `healthy` once the instance passes the first health check.

We can also switch to the “Monitoring” tab to view several different metrics plotted against time, including the number of healthy and unhealthy targets, the

we can also switch to the `Monitoring` tab to view several different metrics plotted against time, including the number of healthy and unhealthy targets, the number of requests, etc.

Note: It might take several minutes after the instances are launched for the health statuses and monitoring data to be available.

Access the load balancer

When we access the load balancer DNS, it should redirect the traffic to the registered instances. Let's verify this by following the steps below:

1. Ensure that the status of the registered target instances is `healthy`.
2. From the sidebar, click "Load Balancers" under the "Load Balancing" option.
3. Select `MyLoadBalancer` from the list.
4. From the bottom panel, copy the DNS name under the "Details" section.
5. Paste the DNS in the URL bar of a new browser tab to make a request to the URL.

You should see our custom hello message, which includes the hostname of the instance your request was directed to. Try making some more requests by refreshing the page, and you should see the hostname changing as the load balancer directs traffic to different instances.

Simulate an application failure

Application Load Balancers, as the name implies, operate at the application level. This means that the load balancer conducts frequent checks to monitor the status of the application alongside the status of the instance itself.

When the application running on our instance fails the health checks that were configured, the instance should be replaced by a fresh instance. Let's simulate an application failure by connecting to one of the instances and manually stopping the application running on it:

1. From the sidebar, click "Instances" under the "Instances" option.
2. Select any one of the running instances, then click the "Connect" button.
3. Click "Connect" to open the EC2 Instance Connect terminal in a new browser window.
4. Run the following command in the terminal. This stops the `httpd` server that is running on the instance.

```
sudo systemctl stop httpd.service
```

The server running on the instance has been shut down, but the instance itself is still running. Regardless, the instance should fail the load balancer health checks and should be replaced with a new instance.

Let's view the status of the registered targets once again. Return to the details page of our `LoadBalancerTargetGroup` target group. After a few minutes have passed, the health status of one of the instances should change to `Unhealthy` once it fails two consecutive health checks. The status should then change to `Draining` as it gets deregistered and taken out of service. Additionally, a new instance should launch, and its status should change to `Healthy` once it passes the health checks.

Clean Up

In this task, we'll delete all the resources we created in this lab. We'll go in the reverse order of creation, starting with the Auto Scaling group.

Delete the Auto Scaling group

Follow the steps below to delete the ASG:

1. Navigate to the "Auto Scaling Groups" page by clicking the "Auto Scaling Groups" option.
2. Select `MyAutoScalingGroup` from the list.
3. Click the "Actions" button and then select the "Delete" button from the drop-down menu.
4. Enter `delete` in the confirmation pop-up, then click the "Delete" button.

Note: Once the ASG is deleted, all instances launched by it will also be terminated. This process can take up to ten minutes.

Delete the launch template

Next, let's delete the instance launch template we used for our ASG. Follow the steps below:

1. Navigate to the "Launch Templates" page by clicking the "Launch Templates" option.
2. Select `AutoScalingLaunchTemplate` from the list of templates.
3. Click the "Actions" button, then click "Delete template."
4. Enter `Delete` in the confirmation pop-up, then click the "Delete" button.

Delete the load balancer

Let's delete the load balancer. Follow the steps below:

1. Navigate to the "Load Balancers" page by clicking the "Load Balancing" option.
2. Select `MyLoadBalancer` from the list of load balancers.
3. Click the "Actions" button, then click "Delete load balancer."
4. Enter `confirm` in the confirmation pop-up, then click the "Delete" button.

Delete the target group

Let's also delete the target group. Follow the steps below:

1. Navigate to the "Target Groups" page by clicking the "Load Balancers" option.
2. Select `LoadBalancerTargetGroup` from the list of target groups.
3. Click the "Actions" button, then click "Delete."
4. Click the "Yes, delete" button in the confirmation pop-up.

Delete the security groups

Finally, let's delete both the security groups that we configured at the start of this Cloud Lab:

1. Navigate to the "Security Groups" page under "Network & Security."
2. Select both `LoadBalancerSecurityGroup` and `LaunchTemplateSecurityGroup` from the list of security groups.
3. Click the "Actions" button, then click "Delete security group."
4. Click the "Delete" button to confirm deletion.